Lightweight Telemetry for FlashInfer JIT and Decode Profiling

Yuzhi Han
Rice University
Houston, Texas, USA
yh165@rice.edu

Abstract

FlashInfer is a high-performance GPU kernel library for large language model (LLM) inference. Its optimized kernels can make steady-state decode latency extremely small, but this also makes system-level overheads such as just-in-time (JIT) compilation, module loading, and decode planning harder to reason about from end-to-end benchmark numbers alone. This project adds a lightweight observability layer to FlashInfer and uses it to profile JIT and decode execution paths. The implementation records structured JSONL telemetry events for JIT cache lookup, compilation, shared-object loading, batch decode planning, and batch decode execution. A workflow benchmark and analysis scripts then aggregate these events into report-ready tables and figures. On an NVIDIA L40S GPU, inclusive telemetry event durations are concentrated in JIT build and decode planning events, while steady-state `decode_run` events average only 0.025 ms. These results show that FlashInfer's decode kernels are very fast once compiled, while observability is especially valuable for identifying first-call, planning, and specialization-related overheads.

Keywords

FlashInfer, LLM inference, observability, profiling, JIT compilation, GPU kernels

1 Introduction

Modern LLM serving systems need low latency, high throughput, and predictable behavior under changing workloads. FlashInfer addresses the kernel-performance side of this problem by providing optimized attention and inference kernels for GPUs. However, practical inference performance is not only determined by kernel runtime. The first invocation of a new specialization may trigger JIT compilation; a decode wrapper may spend time in planning; and different tensor shapes can produce different compilation and cache behavior. Without structured observability, these costs are easy to mix together in a single latency number.

The goal of this project is to make the FlashInfer execution path more observable without relying on heavyweight external profilers for every experiment. Instead of replacing tools such as NVIDIA Nsight Systems or PyTorch Profiler, the project adds a small built-in telemetry layer that can answer common questions during benchmarking: which path triggered JIT compilation or module loading, whether the JIT cache hit or missed for a particular specialization, how much time was spent in decode planning versus decode execution, and how steady-state decode latency changes with `head_dim` or KV cache length.

The main contribution is a project-specific profiling workflow for FlashInfer: instrumentation in selected FlashInfer code paths, benchmark scripts that trigger those paths, JSONL log analysis, and reproducible plots for the final report.

Table 1: Instrumented paths and telemetry measurements.

Path	Events / measurements	Purpose
JIT	cache hit/miss, build, compile count, load	Attribute specialization overhead
Decode wrapper	plan count/time, run count/time	Separate planning and execution

2 Design Goals and Methodology

The design follows four principles. First, telemetry should be lightweight and optional. The system is controlled by environment variables for enablement, sampling rate, output path, and output format. When telemetry is disabled, instrumentation returns early and avoids logging work.

Second, events should be structured rather than free-form text. Each telemetry record is a JSON object containing an event name, timestamp, module name, optional CPU/GPU duration, and event-specific metadata such as specialization name, device architecture, backend, batch size, head dimension, and KV layout.

Third, the selected instrumentation points should cover both one-time setup costs and steady-state execution. Table 1 summarizes the two paths targeted by this project.

Fourth, telemetry logs should be easy to process in an HPC environment. The analysis script is designed to match the JSONL schema emitted by the telemetry layer, then writes both a raw flattened CSV and a per-event summary CSV. Plotting is treated as a downstream report-generation step rather than part of the telemetry design itself.

3 Implementation

3.1 Telemetry Layer

The telemetry layer exposes two main APIs. The first records point events such as JIT cache hits or wrapper call counts. The second records the duration of scoped operations such as JIT build or shared-object loading.

For CPU timing, the implementation uses `time.perf_counter()`. When CUDA is available, `timed_section` also attempts to create CUDA events to record GPU elapsed time. Events are buffered and flushed to a JSONL file at process exit or when the buffer reaches a fixed size. This keeps the logging interface simple while avoiding per-event file writes.

3.2 JIT Instrumentation

JIT instrumentation is inserted in FlashInfer's JIT core path. The implementation records five events: `jit_cache_hit`, `jit_build`, `jit_compile_count`, `jit_load`, and `jit_load_count`. Each JIT record includes a specialization name (`spec_name`) and the CUDA

Table 2: Benchmark environment.

Component	Value
Available GPUs	NVIDIA L40S
Compute capability	8.9 (sm_89)
PyTorch	2.6.0
CUDA	12.6
FlashInfer	Version 0.6.7 with telemetry injection
Main dtype	`torch.float16`
Telemetry format	JSONL

architecture (`device_arch`) when available. This is important because FlashInfer specializes kernels by datatype, head dimension, positional encoding mode, and other compile-time parameters.

3.3 Decode Instrumentation

Decode instrumentation is inserted in the batch decode wrapper path. The most important events are `plan_call_count`, `decode_plan`, `run_call_count`, and `decode_run`. The workflow benchmark uses the paged-KV batch decode wrapper's plan and run calls rather than only single decode. This matters because single-decode benchmarks measure kernel latency and JIT behavior, but they do not trigger wrapper-level decode plan/run telemetry.

3.4 Benchmark and Analysis Workflow

The benchmark workflow is organized as a single coherent execution path rather than a collection of unrelated microbenchmarks. The setup stage configures the Python environment, CUDA-related cache directories, FlashInfer's workspace base, and telemetry defaults. The benchmark stage then runs a workflow script with four phases: a cold single-decode call, repeated warm single-decode calls, one paged-KV batch decode plan call, and repeated paged-KV batch decode run calls.

This design makes the telemetry log easier to analyze: the log corresponds to one intended workflow, and the event summary can be interpreted alongside the phase-level CSV written by the benchmark.

4 Experimental Setup

The benchmark suite uses synthetic random tensors. This isolates kernel and wrapper behavior from model-level effects such as tokenizer overhead, network scheduling, or application-level batching. Because the available hardware was limited to V100 and L40S GPUs, the experiments do not cover FlashInfer APIs that require Blackwell-class SM100-or-newer devices.

5 Results

5.1 Workflow Phase Timing

The single-decode cold call is approximately 1.70×10^5 times slower than the warm single-decode average. This ratio reflects a cold-start contrast rather than an application-level speedup, since the first call includes compilation, loading, and synchronization overhead. Similarly, the batch decode plan call is approximately 1446 times larger than the average measured batch decode run call. These phase-level measurements separate setup costs from warm execution costs in the FlashInfer decode workflow.

Table 3: Phase-level measurements from `bench_workflow.py`.

Phase	Metric	Value ms
`single_decode_cold`	first call	3905.07
`single_decode_steady`	average call	0.0230
`batch_decode_plan`	plan call	1735.09
`batch_decode_run`	average call	1.2003

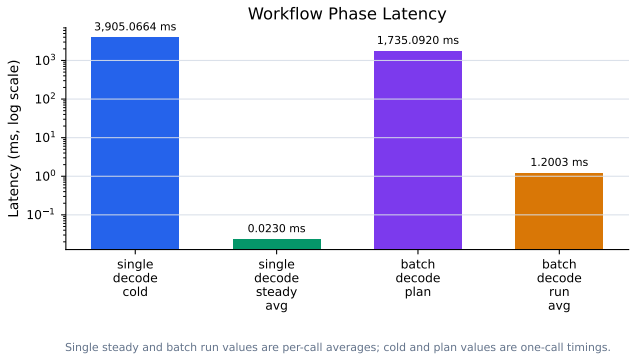

Figure 1: Workflow phase latency. Single steady and batch run values are per-call averages; cold and plan values are one-call timings.

Table 4: Inclusive event-scope timing summary from the final workflow telemetry run. Percentages are shares of summed event durations, not a disjoint wall-clock decomposition.

Event	Count	Total ms	Mean ms	Share %
`jit_build`	2	5262.58	2631.29	74.98
`decode_plan`	1	1735.01	1735.01	24.72
`jit_load`	2	20.24	10.12	0.29
`decode_run`	25	0.63	0.025	0.009

5.2 Event-Level Telemetry Breakdown

The timing summary reports inclusive event-scope durations for the single workflow log. A duration belongs to the full scope recorded by that event. For example, `decode_plan` is interpreted as the complete batch decode planning call, even if lower-level JIT activity occurs inside that call.

Setup-oriented events dominate the timed telemetry records. The two `jit_build` events correspond to the two specializations triggered by the workflow: one for single decode and one for batch decode. Together they sum to 5.26 s. The full `decode_plan` event contributes 1.74 s, while the 25 `decode_run` events sum to 0.63 ms. Thus, the measured workflow separates multi-second setup costs from sub-millisecond repeated wrapper execution.

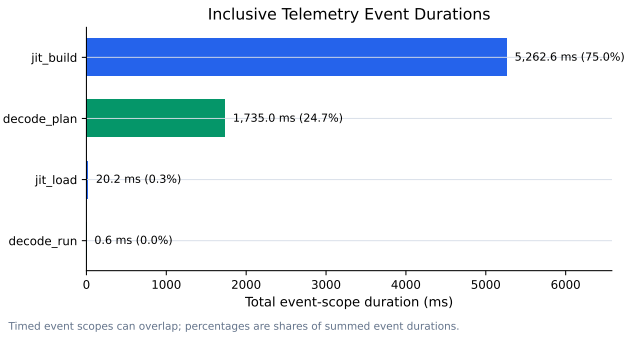

Figure 2: Inclusive event-scope telemetry durations aggregated over the full workflow. Non-timed count events are excluded.

Figure 3: Cold first-call and warm steady-state single-decode latency on a logarithmic scale.

5.3 JIT First-Call Overhead

The workflow compares the first single-decode call to repeated warm single-decode calls for `kv_len=2048`, `num_heads=32`, and `head_dim=128`. The first call latency is 3905.07 ms, while the steady-state average latency is 0.0230 ms. The resulting first-call to steady-state ratio is approximately 1.70×10^5.

The ratio is mathematically correct: 3905.07 ms divided by 0.0230 ms is approximately 1.70×10^5. However, it should not be interpreted as a general application speedup. The numerator is a cold first-call measurement that includes JIT compilation, loading, and CUDA synchronization, while the denominator is a steady-state per-call kernel measurement. This motivates separate reporting of cold-start and warm-start latency.

5.4 JIT Cost by Specialization

FlashInfer compiles different specializations for different operator configurations. The telemetry captures the `spec_name` for each JIT event, allowing the analysis to show which specializations incurred build-path cost.

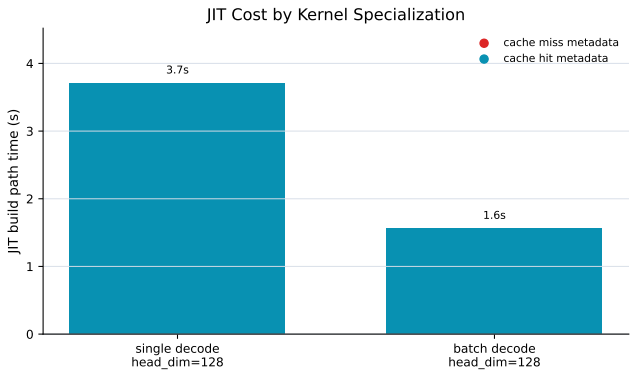

Figure 4: JIT build-path cost by kernel specialization. Colors indicate whether the associated cache lookup metadata reported a hit or miss.

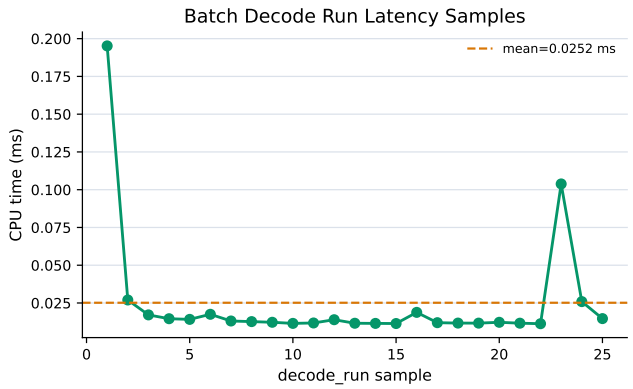

Figure 5: Telemetry samples for `decode_run` events in the batch decode wrapper benchmark.

The workflow run includes two JIT build-path records. Both cache lookup records reported hits, but the `jit_build` scope still captures build-path or validation overhead before the generated shared object is loaded. Specialization-level metadata separates the single-decode specialization from the batch-decode specialization, avoiding a single aggregate setup-time measurement.

5.5 Paged-KV Batch Decode Wrapper Runtime

The wrapper benchmark uses FlashInfer's paged-KV batch decode wrapper with batch size 8, KV length 2048, page size 16, 32 heads, and head dimension 128. The benchmark-level average paged-KV batch decode runtime was 1.2003 ms. The telemetry-level `decode_run` events averaged 0.025 ms, with a minimum of 0.011 ms and a maximum of 0.195 ms. The experiment covers the paged-KV batch decode wrapper path, but it does not cover every FlashInfer batch, GEMM, or Blackwell-specific API.

The benchmark-level average and telemetry-level event average measure different scopes. The benchmark times the full repeated wrapper call region with CUDA synchronization around the measurement loop, while telemetry measures Python-side wrapper

sections per call. The benchmark-level number approximates user-visible latency; the telemetry event supports attribution of internal code-path costs.

6 Discussion

The results support three observations. First, inclusive event-scope telemetry shows that setup work is much larger than steady-state decode execution for this benchmark workflow. This is expected for systems with highly optimized steady-state kernels: once the kernel exists and decode state is planned, repeated decode calls are very small. The current event definitions intentionally preserve high-level semantics, so `decode_plan` is reported as the full planning call rather than being rewritten into a derived subcomponent.

Second, shape specialization matters. The workflow triggers separate JIT specializations for single decode and batch decode. Without `spec_name` in the logs, it would be difficult to connect a build cost to the operator path that caused it.

Third, built-in telemetry complements external profilers. Nsight and PyTorch Profiler can provide deeper kernel timelines, but they are heavier to run and analyze. The telemetry layer gives a low-friction summary that can be collected during ordinary benchmark runs and used to decide where deeper profiling is needed.

There are also limitations. The current decode telemetry records CPU duration for `decode_plan` and `decode_run`, but not GPU event duration for those wrapper events. Adding a GPU timing mode for decode could make CPU/GPU attribution more complete. The current benchmark uses synthetic tensors and a narrow set of shapes, so it does not fully represent an end-to-end serving workload. Hardware availability also limits API coverage: V100 and L40S do not support SM100+ Blackwell-only paths such as TGV GEMM kernels, so those APIs are outside the scope of this evaluation. Finally, the instrumentation is focused on JIT and decode; prefill, sampling, and distributed communication are natural extensions.

7 Conclusion

This project implemented a lightweight telemetry layer and workflow benchmark for profiling FlashInfer JIT and decode execution. The resulting logs make several normally hidden costs explicit: whether a specialization enters the JIT build path, how long shared-object loading takes, when batch decode planning occurs, and how small repeated decode-run sections are after setup. On the tested L40S GPU, this observability shows a clear separation between multi-second setup events and sub-millisecond repeated decode execution.

The results also illustrate why observability is useful for highly optimized inference libraries. FlashInfer's design achieves very low warm decode latency, but that performance can make setup overheads difficult to interpret from end-to-end timings alone. Structured telemetry connects those overheads to specific events and specializations, making cold-start behavior, planning cost, and steady-state execution visible in a single benchmark workflow. This suggests that lightweight built-in telemetry can complement external profilers by providing inexpensive, reproducible diagnostics during routine performance experiments.

References

[1] FlashInfer Project. FlashInfer GitHub repository. https://github.com/flashinfer-ai/flashinfer.

[2] FlashInfer Project. FlashInfer documentation. https://docs.flashinfer.ai.

[3] PyTorch. PyTorch profiler documentation. https://pytorch.org/tutorials/recipes/recipes/profiler.html.

[4] NVIDIA. NVIDIA Nsight Systems. https://developer.nvidia.com/nsight-systems.

[5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of SOSP*, 2023.